

NeurIPS 2023 | Removing the Trickiest Knob in Private Deep Learning

AWS AI and UC Santa Barbara introduce automatic clipping: a one-line replacement that deletes the clipping threshold from any DP optimizer while matching or beating tuned state of the art.

Training a neural network with a differential-privacy (DP) guarantee follows a well-worn recipe. For each example in a batch you compute its own gradient, rescale that gradient so its norm never exceeds a fixed threshold R , add Gaussian noise calibrated to R , and only then take an optimizer step. This per-sample clipping is what bounds any single example's influence on the model, and it is what makes the privacy accounting go through. It also introduces R , a hyperparameter that turns out to be one of the most consequential and least forgiving choices in the entire pipeline.

The trouble is that R behaves nothing like a gentle knob. Set it a little too high and the noise, which scales with R , drowns the signal; set it too low and every gradient is squashed to the same tiny norm. Because the right value is task- and model-specific, practitioners end up searching jointly over R and the learning rate, a two-dimensional grid that, for large models, can take days to months of compute. Worse, that search reads the private training data, so it quietly spends extra privacy budget on top of the training itself.

This paper, "Automatic Clipping: Differentially Private Deep Learning Made Easier and Stronger" from AWS AI and UC Santa Barbara, makes a clean claim: R does not need to be tuned, because it does not need to exist. The authors show the threshold can be removed from any DP optimizer with essentially a one-line code change, yielding a method that is just as private, just as fast, and at least as accurate as the best hand-tuned baselines, with a convergence guarantee that matches ordinary SGD.

Paper: <https://arxiv.org/abs/2206.07136> | Code: <https://github.com/awsmlabs/fast-differential-privacy>

Why the clipping threshold is such a headache

Standard DP-SGD, following Abadi et al., clips each per-sample gradient g by the factor $\min(R / \|g\|, 1)$: gradients longer than R are scaled down to length R , and shorter ones are left alone. In practice, though, the thresholds that give the best accuracy are tiny, small enough that essentially every per-sample gradient is longer than R and therefore gets clipped at every step. When that happens the min collapses to its first branch, $R / \|g\|$, for almost every example. The exact value of R stops carrying any per-example magnitude information; it acts only as a global scale. That observation is the crack the paper pries open.

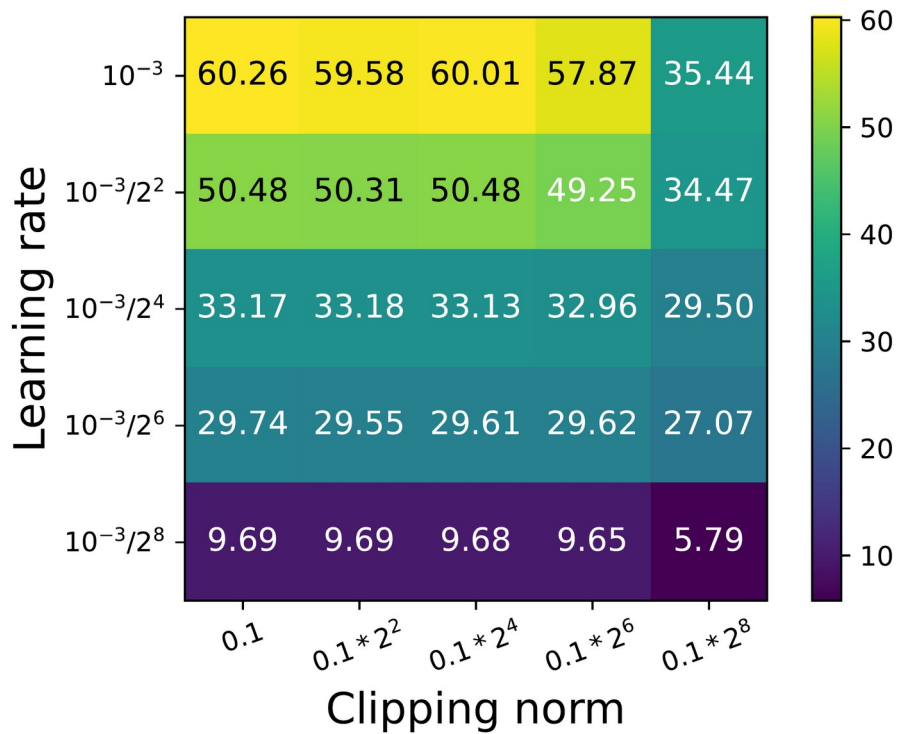


Figure 1. Test accuracy of ResNet18 on ImageNet under DP-SGD, swept over the clipping norm (x-axis) and learning rate (y-axis). Accuracy runs from about 60% at a good configuration down to single digits, and even along a strong learning-rate row it falls sharply once the clipping norm is set too large. The threshold, not just the learning rate, decides the outcome.

From a threshold to a normalization: AUTO-V

If nearly every gradient is being scaled by $R / \|g\|$ anyway, why keep the min at all? Automatic clipping drops it and applies the normalization to every gradient unconditionally. The paper calls this vanilla variant AUTO-V:

$$\text{Clip_AUTO-V}(g; R) = R / \|g\|.$$

Normalizing every gradient to the same length has a concrete benefit: it maximizes the alignment between the aggregated private gradient and the true gradient, because no single large example can dominate the sum. Figure 2 makes this visible. The per-step dot product between the private and true gradient stays markedly higher for AUTO-V than for Abadi's clipping throughout training.

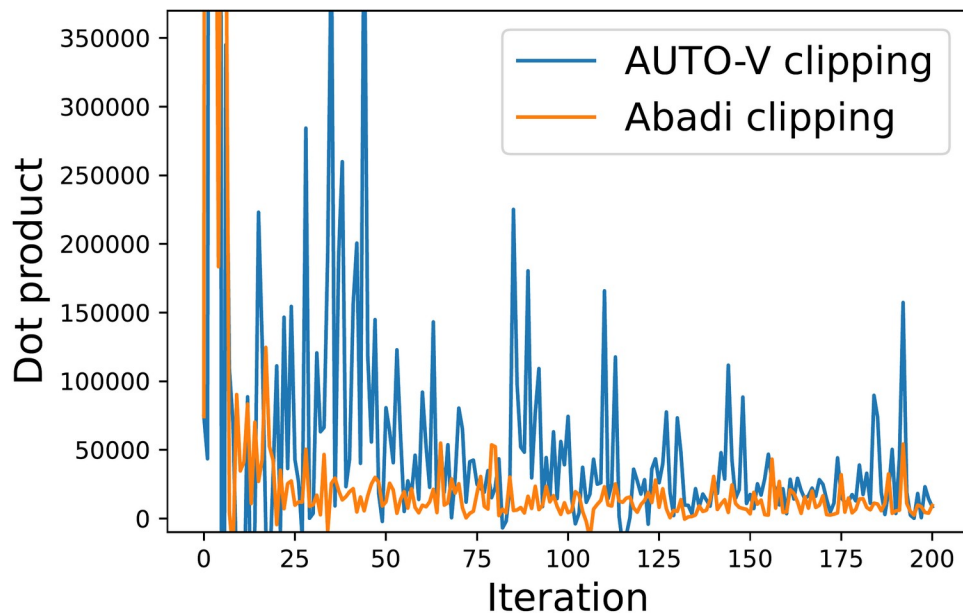


Figure 2. Per-step dot-product similarity between the private (clipped, noised) gradient and the true gradient over training. AUTO-V normalization (blue) keeps the private gradient far more aligned with the true gradient than Abadi's clipping (orange), which is the intuition behind dropping the threshold and simply normalizing.

But pure normalization overshoots. If every gradient is forced to the same magnitude, then small gradients, the ones near a good solution where you want the updates to shrink, are inflated back to full size. Training enters a "lazy region" where it can no longer settle, and AUTO-V on its own can fail to converge.

Restoring magnitude: AUTO-S and the disappearing threshold

The fix is small and surgical. Instead of dividing by $\|g\|$, AUTO-S divides by $\|g\| + \gamma$ for a small positive stability constant γ : $\text{Clip_AUTO-S}(g; R) = R / (\|g\| + \gamma)$. For a large gradient the γ is negligible and AUTO-S behaves like AUTO-V; but as a gradient shrinks toward zero, the clipped result approaches g / γ rather than a fixed length, so small gradients stay small. That single term restores the magnitude information the normalization threw away, and it is what lets the method converge all the way to a zero-gradient solution.

The last step is where the threshold actually disappears. Multiplying every clipped gradient by a constant R is mathematically identical to leaving R at 1 and scaling the learning rate instead, and the noise scale rides along with it, so the privacy guarantee is untouched. The authors therefore simply fix $R = 1$. What remains is an optimizer with no clipping threshold to choose: one default stability constant (the paper uses $\gamma = 0.01$) replaces an entire axis of the hyperparameter grid, and the same substitution drops into DP-SGD, DP-Adam, or DP-LAMB, and into existing libraries such as Opacus, by swapping a single clipping function.

Does deleting the knob cost accuracy? No.

The central experiment asks whether a fixed, threshold-free rule can really match a carefully tuned R . Figure 3 answers it directly. Each heatmap cell is the SST-2 test accuracy of

RoBERTa-base at $\epsilon = 3$ for a given (clipping norm, learning rate) pair; the extra AUTO-S column on the right uses no tuned threshold at all. It lands on the same accuracy as the best hand-tuned R, which means you can search the learning rate alone and skip the R axis entirely, cutting the tuning cost roughly 5 times (and in the privacy budget that tuning would spend).

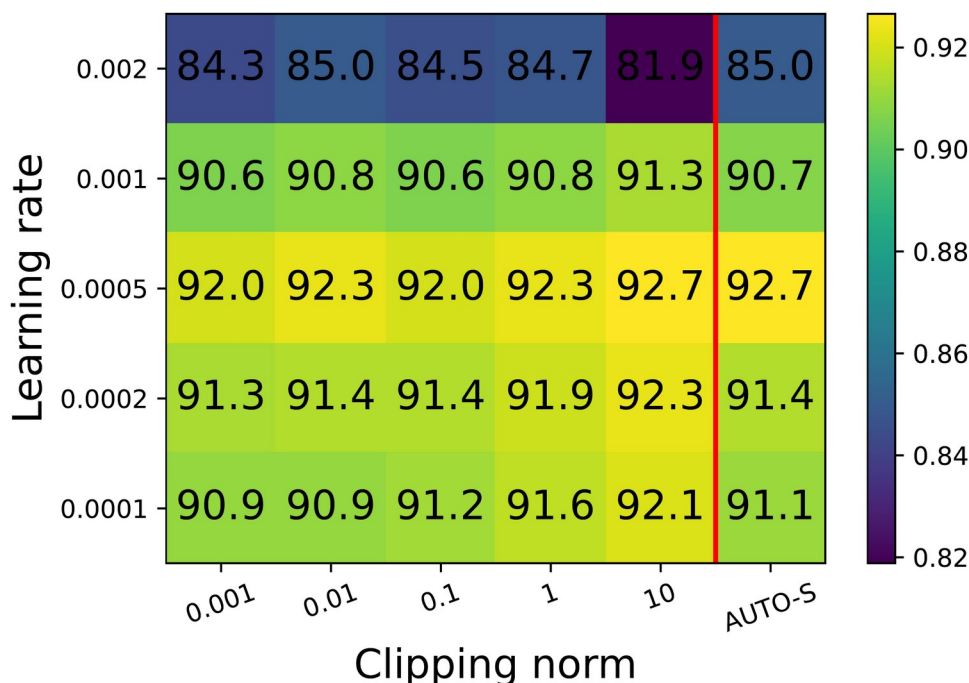


Figure 3. RoBERTa-base test accuracy on SST-2 (DP-Adam, $\epsilon = 3$) across clipping thresholds R and learning rates. The AUTO-S column past the red line uses no tuned threshold, yet reaches the same accuracy as the best hand-tuned R; searching only the learning rate shrinks the hyperparameter grid to about a fifth.

The pattern holds across tasks, and often the gap is not just "matched" but clearly better. On image benchmarks AUTO-S improves over Abadi's clipping at the same privacy budget on every task the paper reports, with the largest jumps where accuracy has the most room to move.

Table 1. AUTO-S versus the tuned baseline at matched privacy budget ϵ . Numbers are test accuracy (%); higher is better.

Task (model)	ϵ	Baseline (Abadi / prior)	AUTO-S
MNIST (4-layer CNN)	3	98.15	99.11
CIFAR-10 (SimCLRv2)	2	92.70	94.42
ImageNette (ResNet9)	8	60.71	71.11
SST-2 (RoBERTa-base)	3	91.86	92.32
SST-2 (RoBERTa-large)	8	93.81	94.61

The language results tell the same story. On the GLUE tasks with RoBERTa, AUTO-S matches or beats the previous best DP fine-tuning numbers, and on E2E table-to-text generation with GPT2 it reaches a BLEU of 64.18 (GPT2-large, $\epsilon = 3$), at the top of the reported models. The recurring theme is that the method never needs a tuned threshold to get there.

Why it works, not just that it works

Automatic clipping is not only an empirical trick; the paper proves it converges. For non-convex objectives, automatic DP-SGD with AUTO-S drives the minimum expected gradient norm down at the rate $O(T^{-1/4})$ in the number of steps T , the same asymptotic rate as standard, non-private SGD. Figure 4 traces the upper bound on that quantity across several values of a gradient-asymmetry ratio r , and every curve settles onto the standard-SGD slope. The analysis also shows the method is essentially insensitive to γ : any positive value yields the same asymptotic rate, which is what justifies shipping a single default instead of yet another thing to tune.

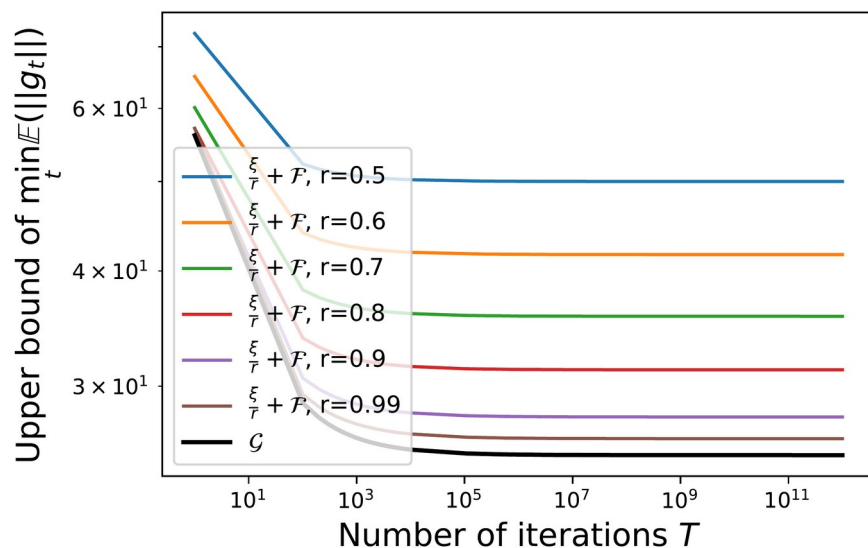


Figure 4. Convergence of automatic DP-SGD (AUTO-S): the upper bound on $\min_t \mathbb{E}(\|g_t\|)$ against the number of iterations T on a log-log scale, for several values of the gradient-asymmetry ratio r . Every curve flattens at the standard-SGD rate $O(T^{-1/4})$, so removing the threshold costs nothing asymptotically.

Takeaway

The clipping threshold has been treated as a critical, must-tune knob since DP deep learning began. The message of automatic clipping is that it can simply be removed: normalize each per-sample gradient, add a tiny stability constant, fix the threshold to one, and you are left with an optimizer that is as private, as fast, and as accurate as the best hand-tuned DP methods, backed by a convergence guarantee matching SGD. The honest boundary is that automatic clipping changes how gradients are clipped, not the underlying privacy-utility trade-off of DP itself: noise still has to be added, and very tight privacy budgets are still hard. What it removes is a large, expensive, and largely unnecessary source of friction, which is enough to make private training feel a lot more like ordinary training.

Because the change is a drop-in replacement for the clipping step, it costs almost nothing to try: the authors release it in the open-source fast-differential-privacy library, where switching an existing DP-Adam or DP-SGD run to automatic clipping means swapping one function and deleting the threshold from the search. For most practitioners, that turns a two-dimensional hyperparameter hunt into a one-dimensional one, with no measured cost in accuracy or privacy.